

State Pack: A Content-Addressed Protocol for Stateless Transformer Inference

M. Sadiq

State Pack Project

<https://github.com/mauludsadiq/State-Pack>

May 5, 2026

Abstract

We present **State Pack**, a content-addressed protocol for efficient transformer inference in multi-step agent workloads. State Pack treats KV cache state as a portable, verifiable artifact rather than ephemeral server-side session data. The core abstraction is a pure function: $(\text{state_hash}, \text{delta}) \rightarrow (\text{new_state_hash}, \text{output})$, where `state_hash` is the SHA-256 address of a serialized `past_key_values` tensor. This design eliminates redundant prefill computation across agent steps, producing 90–95% token reduction across GPT-2 (124M), Qwen2.5-3B, and Mistral-7B-Instruct on 20-step agent loops. A split Rust/Python implementation achieves 9.1ms protocol latency alongside 47ms model inference on CPU. All results are fully reproducible: `pip install state-pack`.

1 Introduction

Autoregressive transformer models process tokens sequentially. At inference time, the attention mechanism requires access to all prior key-value pairs — the KV cache. In agent loops, this cache grows with every step: a 40-step loop with a 500-token base prompt and 30-token deltas requires processing [500, 530, 560, ..., 1670] tokens at steps 1 through 40. Total naive cost: $\sim 43,000$ tokens. Total *necessary* computation: $\sim 1,700$ tokens (base once, plus deltas).

KV cache state computed for a given input prefix is deterministic and reusable. If the same base prompt is processed by the same model, the resulting `past_key_values` tensor is identical. This makes KV state *content-addressable*: the SHA-256 hash of the input text uniquely identifies the KV state it produces.

1.1 Contributions

1. A formal stateless inference protocol: $(\text{state_hash}, \delta) \rightarrow (\text{new_state_hash}, y)$
2. Content-addressed KV cache serialization with float16 quantization (50% size reduction, zero top-1 accuracy loss, verified experimentally)
3. A COMPACT primitive preventing unbounded delta chain growth
4. A GC command for bounded storage in production deployments
5. A split Rust/Python implementation: 9.1ms protocol latency, $12\times$ faster than the Python baseline
6. Empirical evaluation across three model architectures showing 90–95% token reduction

2 Related Work

Prefix caching in inference engines. vLLM’s PagedAttention [1] manages KV cache memory efficiently but does not expose portable, addressable cache state across requests. SGLang’s RadixAttention [2] reuses prefixes within a single server instance. LMCache [3] supports KV offloading to CPU/disk/S3 within vLLM but requires engine-level integration. State Pack differs by treating KV state as a portable protocol artifact independent of any specific inference engine.

Prompt caching in managed APIs. OpenAI, Anthropic, and Google offer prompt caching with 50–75% discounts on cached input tokens. These are server-side optimizations invisible to the client. State Pack operates at the client layer, producing 90%+ reduction by avoiding transmission of redundant history entirely.

Content-addressed storage. IPFS [4] and Nix [5] demonstrate content-addressable storage as infrastructure primitives. State Pack applies this pattern to ML inference state, enabling deduplication, auditability, and reproducibility not present in session-based inference systems.

3 Protocol Design

3.1 State Representation

A *state packet* consists of:

- `state_hash`: SHA-256(`base_text`)
- `past_key_values`: serialized KV cache tensor (float16, .pt)
- `manifest`: JSON metadata (`model`, `base_tokens`, `blob_sha256`, `packet_id`)
- `packet_id`: SHA-256(`version` | `model` | `base_sha256` | `base_bytes` | `blob_sha256` | `blob_bytes`)

3.2 Core Operations

Create(t) Run base text t through model, serialize `past_key_values`, store blob. Returns $h = \text{SHA-256}(t)$. *Idempotent*: same text always produces same hash.

Infer(h, δ) Load KV state for h . Run δ against cached state. Returns $h' = \text{SHA-256}(h \mid \text{SHA-256}(\delta))$ and output token. Emits tamper-evident receipt.

Compact($h, [\delta_1, \dots, \delta_n]$) *Fold accumulated delta list into fresh base state. Return h_{fresh} .* Prevents unbounded growth of the delta chain.

Gc(`store`, τ) Remove manifest+blob pairs older than τ days or exceeding a maximum count. Keeps storage bounded in production deployments.

3.3 Hash Chain

The client maintains a hash chain:

$$h_0 \xrightarrow{\delta_1} h_1 \xrightarrow{\delta_2} h_2 \longrightarrow \dots \xrightarrow{\text{COMPACT}} h_{\text{fresh}}$$

Each transition is deterministic given the same model weights. The chain constitutes a cryptographic audit trail of inference history.

3.4 Compaction Policy

Without compaction, accumulated delta tokens eventually dominate base tokens, reducing marginal savings. We define two policies:

- **ThresholdPolicy**: compact when $\sum \text{delta_tokens} > \text{base_tokens} \times r$
- **SavingsPolicy**: compact when per-step savings fall below a minimum percentage

Empirically, `ThresholdPolicy(r=1.0, max_steps=20)` maintains >90% savings over 20-step loops with 2 compactions on average.

4 Implementation

4.1 KV Cache Serialization

HuggingFace `past_key_values` is a tuple of (key, value) tensor pairs, one per layer. We serialize via `torch.save` with float16 quantization. Experimental verification (Section 5) confirms zero top-1 accuracy impact across all tested architectures.

`DynamicCache` compatibility (transformers ≥ 4.36) is handled via `DynamicCache.from_legacy_cache()` and `cache.to_legacy_cache()`.

4.2 Split Architecture

The protocol layer is implemented in Rust (Axum framework):

- SHA-256 content addressing and manifest I/O
- Tamper-evident receipt generation
- HTTP routing

The inference layer is implemented in Python:

- Model loading (HuggingFace `AutoModelForCausalLM`)
- KV cache forward pass
- Blob serialization/deserialization

This split eliminates Python GIL contention on the protocol hot path. Protocol latency: 9.1ms (Rust) vs. 107ms (Python baseline), 12 $\times$ improvement.

4.3 Multi-Agent Deduplication

Content addressing naturally deduplicates shared base states. When N agents share the same system prompt, `CREATE` is called once and $N - 1$ subsequent calls are cache hits. Empirically: first agent 0.951s, subsequent agents 0.003s (316 $\times$ faster). At 1,000 concurrent agents sharing a base prompt, the cost reduction compound: base tokens paid once, delta tokens paid per agent per step.

5 Evaluation

5.1 Token Reduction

Table 1 reports token reduction across model families and sizes. Savings are consistent at 90–95%, confirming the reduction is structural (protocol-driven) rather than model-dependent.

Table 1: Token reduction across model families (20-step agent loops).

Model	Params	Naive Tokens	SP Tokens	Reduction
GPT-2	124M	18,660	875	95.3%
Qwen2.5-3B	3B	5,412	495	90.9%
Mistral-7B	7B	6,662	605	90.9%
OpenAI API	—	17,929	1,320	92.6%

5.2 Quantization Accuracy

Float16 vs. float32 KV cache comparison (GPT-2):

- Max logit difference: 0.066 (float16), 0.307 (bfloat16)
- Top-1 token match: **True** (both precisions)
- Blob size reduction: 50%

5.3 Protocol Latency

Table 2: Protocol layer latency: Rust vs. Python.

Implementation	Avg	Min	Max
Python (baseline)	107ms	24ms	190ms
Rust (v0.3)	9.1ms	2.8ms	19.7ms
Improvement	12×	—	—

5.4 Compaction Effectiveness

`ThresholdPolicy(r=0.5,max_steps=10)` over 20 steps: compactions triggered at steps 4 and 14, final savings 90.8%, average step latency 42ms.

5.5 Limitations

1. **CPU-bound speedup.** Wall-clock speedup is 1.4× for 7B models on CPU. GPU inference is expected to yield 3–4× improvement based on GPT-2 results; this remains future work.
2. **KV cache portability.** Cache blobs are architecture-specific. A Mistral-7B blob cannot be loaded into a Qwen2.5 model. Cross-device transfer within the same architecture is not yet implemented.
3. **OpenAI integration mechanism.** The OpenAI benchmark uses structured context discipline (delta-only messages) rather than KV cache transfer, which is unavailable via the OpenAI API. Native KV transfer would further improve savings.
4. **Non-determinism.** Receipts hash inputs only. With temperature > 0, the same receipt may correspond to different outputs. Auditable deployments should use greedy decoding.

6 Discussion

When State Pack helps most. Long agent loops (>10 steps) with stable base prompts; multi-agent workloads with shared system prompts; regulated environments requiring auditable inference; cost-sensitive deployments on frontier models.

When State Pack helps less. Single-turn queries with no prefix reuse; highly variable base prompts; inference providers with aggressive native prompt caching that already eliminates redundant prefill cost.

Relationship to existing systems. State Pack is complementary to vLLM and SGLang, not competitive. A complete production deployment could use State Pack for client-side protocol and KV addressability, with vLLM serving actual inference requests.

7 Conclusion

State Pack demonstrates that treating transformer KV cache as a content-addressed, portable protocol artifact produces consistent 90–95% token reduction across model families from 124M to 7B parameters. The stateless pure-function design enables horizontal scalability, natural multi-agent deduplication, and cryptographic auditability without engine-level modifications. A Rust protocol implementation achieves 9.1ms overhead, making the protocol layer negligible relative to model inference time. All results are reproducible from a single command.

References

- [1] W. Kwon et al., *Efficient Memory Management for Large Language Model Serving with PagedAttention*, SOSP 2023.
- [2] L. Zheng et al., *SGLang: Efficient Execution of Structured Language Model Programs*, 2023.
- [3] LMCache Team, *LMCache: KV Cache Offloading for vLLM*, 2024.
- [4] J. Benet, *IPFS — Content Addressed, Versioned, P2P File System*, 2014.
- [5] E. Dolstra, *The Purely Functional Software Deployment Model*, PhD thesis, Utrecht University, 2006.